

1.1.1 Gestió dels esdeveniments

Per poder reaccionar a un esdeveniment, cal assignar un gestor d'esdeveniments (event handler) a un element del DOM. Hi ha diverses maneres de fer-ho:

A través d'un atribut HTML (no recomanat):

```
1 <button onclick="alert('Hola!')">Fes clic</button>
```

Aquesta és la manera més antiga i menys recomanada, ja que barreja HTML i JavaScript, dificultant la mantenibilitat.

Assignació directa via JavaScript

```
1 let boto = document.getElementById("meuBoto");
2 boto.onclick = function() {
3   alert("Has fet clic!");
4 };
```

Aquesta manera substitueix qualsevol altre onclick anterior. Només es pot assignar **un únic gestor** per cada tipus d'esdeveniment.

Amb addEventListener() (la més moderna i recomanada)

```
1 let boto = document.getElementById("meuBoto");
2 boto.addEventListener("click", function() {
3   alert("Clic capturat amb addEventListener");
4 });
```

Aquesta forma és la més flexible, perquè:

- Permet **afegir múltiples gestors** per un mateix esdeveniment.
- Separa clarament el **comportament del contingut HTML**.
- És compatible amb més funcionalitats com **captura**, **delegació**, etc.

Exemple complet

En aquest exemple crearem una petita pàgina amb diversos elements que responen a diferents tipus d'esdeveniments: un botó que mostra un missatge en fer clic, una caixa que canvia de color quan el ratolí hi passa per sobre, un camp de text que detecta quan s'escriu alguna cosa (keyup) i un altre botó amb resposta al doble clic. L'objectiu és veure com **assignar diferents gestors d'esdeveniments** a diferents elements, i entendre com funciona l'objecte event.

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
```

```
6     <meta name="viewport" content="width=device-width, initial-
      scale=1.0">
7     <title>Esdeveniments</title>
8 </head>
9
10 <body>
11     <button id="btnClick">Fes clic</button>
12     <button id="btnDoble">Doble clic</button>
13     <div id="caixa" style="width:150px; height:150px; background
      :#ddd; margin:10px;"></div>
14     <input type="text" id="campText" placeholder="Escriu alguna
      cosa..." />
15     <p id="missatge"></p>
16
17     <script>
18         // Botó de clic simple
19         document.getElementById("btnClick").addEventListener("
      click", () => {
20             alert("Has fet un clic!");
21         });
22
23         // Botó amb doble clic
24         document.getElementById("btnDoble").addEventListener("
      dblclick", () => {
25             alert("Has fet un doble clic!");
26         });
27
28         // Caixa que canvia de color amb el ratolí
29         const caixa = document.getElementById("caixa");
30
31         caixa.addEventListener("mouseover", () => {
32             caixa.style.backgroundColor = "lightblue";
33         });
34
35         caixa.addEventListener("mouseout", () => {
36             caixa.style.backgroundColor = "#ddd";
37         });
38
39         // Input que detecta quan s'escriu
40         document.getElementById("campText").addEventListener("
      keyup", function (e) {
41             const text = e.target.value;
42             document.getElementById("missatge").textContent = '
      Has escrit: ${text}';
43         });
44
45     </script>
46 </body>
47
48 </html>
```

En aquest exemple veiem repetidament un **patró per gestionar esdeveniments**:

1. Busquem l'element que rebrà l'acció, per exemple `document.getElementById("btnClick")`
2. Li posem una escolta a l'acció que rebrà `.addEventListener("click",`
3. Finalment, quan es produeixi aquest esdeveniment (click), cridarem a una funció (anònima en aquest cas, però no té per què ser sempre així) `, () =>`

```
{ alert("Has fet un clic!"); };
```

Això és així en els exemples dels `<button>` i `<div>` però en el camp de text observem que la funció anònima té un paràmetre que hem anomenat `e`.

Quan definim un gestor d'esdeveniments, **JavaScript envia automàticament** un objecte que representa l'esdeveniment ocorregut. Podem anomenar aquest paràmetre com vulguem: `e`, `evt`, `event`, etc.

Aquest objecte `event` conté **tota la informació sobre el que ha passat**: quin tipus d'esdeveniment és, en quin element s'ha produït, si hi havia tecles especials premudes, la posició del ratolí, etc.

Per tant, repassant l'exemple, veiem que:

- `e` és l'objecte de l'esdeveniment `keyup`.
- `e.target` apunta al camp de text on s'ha escrit.
- `e.target.value` permet llegir el contingut actual del camp de text.

Així, cada vegada que l'usuari escriu una lletra, el navegador sap **quin element** ha generat l'esdeveniment i n'extreu el valor per actualitzar el paràgraf amb el missatge.

1.1.2 Propagació d'esdeveniments: Bubbling i Capturing

Quan es produeix un esdeveniment al navegador (com fer clic en un element), aquest **no només afecta l'element directament implicat**, sinó que pot **propagarse cap amunt o cap avall per l'estructura del DOM**.

Aquesta propagació segueix dues fases principals:

1. **Capturing (captura)**: l'esdeveniment es propaga **des de l'arrel del document (document) cap al node objectiu**.
2. **Bubbling (bombolla)**: després de l'execució en el node objectiu, l'esdeveniment es propaga **cap amunt, des del node objectiu fins al document**.

Aquesta doble fase permet que **diversos elements d'una jerarquia** puguin **escoltar un mateix esdeveniment** en diferents moments de la seva propagació.

Exemple visual

Imaginem aquesta estructura HTML:

```
1 <div id="pare">
2   <button id="fill">Fes clic</button>
3 </div>
```

Quan es fa clic al botó, l'esdeveniment es comporta així:

1. **Captura (*capturing*)**: document → html → body → div#pare → button#fill
2. **Target**: el botó (#fill) rep l'esdeveniment
3. **Bombolla (*bubbling*)**: button#fill → div#pare → body → html → document

Bubbling: comportament per defecte

Quan utilitzem `addEventListener` sense indicar res més, l'esdeveniment es captura en fase de *bubbling*. Això vol dir que **primer s'executa el gestor de l'element més intern**, i després els dels seus "pares", de dins cap a fora.

A continuació, veurem un exemple de *bubbling*:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Bubbling – comportament per defecte</title>
5   <style>
6     #contenedor { padding: 30px; background: #e5e7eb; border-
7       radius: 10px; }
8     #boto { padding: 10px 16px; }
9   </style>
10 </head>
11 <body>
12   <h2>Bubbling (per defecte)</h2>
13
14   <div id="contenedor">
15     <button id="boto">Fes clic</button>
16   </div>
17
18   <script>
19     const contenedor = document.getElementById('contenedor');
20     const boto = document.getElementById('boto');
21
22     // Listener al botó (fase de bombolla per defecte)
23     boto.addEventListener('click', (e) => {
24       console.log('Clic al botó (bombolla)');
25       // Observa target vs currentTarget
26       console.log('target =', e.target.id, '| currentTarget =',
27         e.currentTarget.id);
28     });
29
30     // Listener al contenedor (fase de bombolla per defecte)
31     contenedor.addEventListener('click', (e) => {
32       console.log('Clic al contenedor (bombolla)');
33       console.log('target =', e.target.id, '| currentTarget =',
34         e.currentTarget.id);
35     });
36   </script>
37 </body>
38 </html>
```

Quan es fa clic al botó, apareix per consola:

```
1 Clic al botó (bombolla)
2 target = boto | currentTarget = boto
3 Clic al contenedor (bombolla)
4 target = boto | currentTarget = contenedor
```

L'esdeveniment **puja** des del botó cap al contenidor.

Capturing: control avançat

Si volem escoltar l'esdeveniment **abans que arribi al seu objectiu final**, podem activar la fase de **captura** afegint `true` com a tercer paràmetre a `addEventListener`:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Capturing – control avançat</title>
5   <style>
6     #contenidor { padding: 30px; background: #e5e7eb; border-
7       radius: 10px; }
8     #boto { padding: 10px 16px; }
9   </style>
10 </head>
11 <body>
12   <h2>Capturing (3r paràmetre = true)</h2>
13
14   <div id="contenidor">
15     <button id="boto">Fes clic</button>
16   </div>
17
18   <script>
19     const contenidor = document.getElementById('contenidor');
20     const boto = document.getElementById('boto');
21
22     // Listener al contenidor en fase de CAPTURA (true)
23     contenidor.addEventListener('click', (e) => {
24       console.log('Capturat pel contenidor (captura)');
25       console.log('target =', e.target.id, '| currentTarget =',
26         e.currentTarget.id);
27     }, true); // <- activa la fase de captura
28
29     // Listener al botó (bombolla per defecte)
30     boto.addEventListener('click', (e) => {
31       console.log('Clic al botó (bombolla)');
32       console.log('target =', e.target.id, '| currentTarget =',
33         e.currentTarget.id);
34     });
35   </script>
36 </body>
37 </html>
```

Ara, la sortida canviarà:

```
1 Capturat pel contenidor (captura)
2 target = boto | currentTarget = contenidor
3 Clic al botó (bombolla)
4 target = boto | currentTarget = boto
```

Primer s'executa el contenidor en **fase de captura**, després el botó en la **fase de bombolla**.

Aturar la propagació

Podem **aturar completament** la propagació d'un esdeveniment amb `event.stopPropagation()` dins del gestor:

```
1 document.getElementById("boto").addEventListener("click", (e) =>
2   {
3     console.log("Només el botó");
4     e.stopPropagation();
5   });
```

Amb això, si es fa clic al botó, l'esdeveniment no arriba al contenidor ni en *bubbling* ni en *capturing*.

Quan és útil cadascuna de les escoltes?

- La **captura** pot ser útil quan volem **interceptar un esdeveniment abans que arribi al seu destí**, com en casos de validació o bloqueig de comportament.
- **bubbling** és ideal per **delegar esdeveniments**: per exemple, en una llista `<ul>`, pots capturar tots els clics dels seus `<li>` des del pare.

1.2 Programació dinàmica d'esdeveniments

Ara que entenem com escoltar esdeveniments, podem entrar en la **programació dinàmica**, que ens permet **crear comportaments complexos i reutilitzables** a partir de com l'usuari interactua amb la pàgina.

Què vol dir "dinàmica"?

Vol dir que podem **assignar, eliminar o modificar gestors d'esdeveniments en temps real**, segons el context de l'aplicació. També vol dir que podem treballar amb **funcions anònimes, funcions externes**, o fins i tot **passar paràmetres** a través dels esdeveniments.

1.2.1 Assignar gestors en temps real

Podem afegir o treure gestors quan calgui:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Assignar gestors en temps real</title>
5   <style>
6     body { font-family: system-ui, -apple-system, sans-serif;
7           padding: 20px; }
8     button { padding: 10px 16px; border-radius: 8px; border: 1px
9             solid #ccc; cursor: pointer; }
10    #clicable { margin-top: 10px; background-color: #f3f4f6; }
11  </style>
```

```
10 </head>
11 <body>
12
13   <h2>Assignar gestors en temps real</h2>
14
15   <p>
16     Fes clic al botó <strong>Afegir esdeveniment</strong> per
17     activar el comportament del botó "Clicable".
18     Passats 5 segons, l'esdeveniment s'eliminarà automàticament.
19   </p>
20
21   <button id="activar">Afegir esdeveniment</button>
22   <button id="clicable">Clicable</button>
23
24   <p id="info"></p>
25
26   <script>
27     // 1. Seleccionem els elements
28     const btnActivar = document.getElementById('activar');
29     const btnClicable = document.getElementById('clicable');
30     const info = document.getElementById('info');
31
32     // 2. Definim la funció que mostrarà el missatge quan es
33     // faci clic
34     function mostrarMissatge() {
35       console.log('Clic rebut!');
36       info.textContent = 'Has fet clic al botó clicable!';
37     }
38
39     // 3. Afegim el gestor en temps real
40     btnActivar.addEventListener('click', () => {
41       // Afegim l'esdeveniment click al botó "clicable"
42       btnClicable.addEventListener('click', mostrarMissatge);
43       info.textContent = 'Esdeveniment activat! (durant 5 segons)';
44     });
45
46     // Després de 5 segons, eliminem el listener
47     setTimeout(() => {
48       btnClicable.removeEventListener('click', mostrarMissatge);
49       info.textContent = 'Esdeveniment desactivat.';
50     }, 5000);
51   </script>
52 </body>
53 </html>
```

Això és molt útil, per exemple, per **activar** o **desactivar funcionalitats** segons l'estat del programa.

1.2.2 Passar paràmetres i fer servir el context

Com ja vàrem observar en un apartat anterior, els gestors d'esdeveniments reben com a primer paràmetre un objecte especial anomenat `event`, que conté informació sobre què ha passat:

```
1 document.addEventListener("keydown", function(e) {
2   console.log("Has premut la tecla:", e.key);
3 });
```

També podem usar funcions que **acceptin arguments addicionals**, usant funcions fletxa o anònimes com a embolcalls:

```
1 function saluda(nom) {
2   alert("Hola, " + nom);
3 }
4
5 boton.addEventListener("click", () => saluda("Júlia"));
```

1.2.3 Delegació d'esdeveniments

Una tècnica molt poderosa consisteix a **assignar l'esdeveniment a un element pare**, i deixar que aquest gestioni els esdeveniments dels fills. Això és útil quan els elements fills **es generen dinàmicament**, com en una llista que creix amb el temps:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
4   <title>Delegació d'esdeveniments</title>
5   <style>
6     body {
7       font-family: system-ui, -apple-system, sans-serif;
8       padding: 20px;
9     }
10    #llista {
11      list-style: none;
12      padding: 0;
13      margin-top: 10px;
14    }
15    #llista li {
16      background: #e5e7eb;
17      margin: 4px 0;
18      padding: 8px 12px;
19      border-radius: 6px;
20      cursor: pointer;
21      transition: 0.2s;
22    }
23    #llista li:hover {
24      background: #dbeafe;
25    }
26    button {
27      padding: 8px 12px;
28      border-radius: 6px;
29      border: 1px solid #ccc;
30      cursor: pointer;
31      background: #f9fafb;
32    }
33  </style>
34 </head>
35 <body>
```



```
36
37 <h2>Delegació d'esdeveniments</h2>
38
39 <p>
40   En aquest exemple, només el <strong>&lt;ul&gt;</strong> té
      un gestor d'esdeveniments.
41   Cada vegada que fem clic en un element <strong>&lt;li&gt;</strong>, el gestor del pare detecta quin fill s'ha clicat
      gràcies a <em>e.target</em>.
42 </p>
43
44 <button id="afegir">Afegir nova tasca</button>
45
46 <ul id="llista">
47   <li>Tasca 1</li>
48   <li>Tasca 2</li>
49   <li>Tasca 3</li>
50 </ul>
51
52 <script>
53   const llista = document.getElementById('llista');
54   const botoAfegir = document.getElementById('afegir');
55   let comptador = 3; // per numerar les noves tasques
56
57   // Delegació: escoltem un sol cop al <ul>
58   llista.addEventListener('click', function(e) {
59     // Comprovem si el clic ha estat sobre un <li>
60     if (e.target.tagName === 'LI') {
61       alert('Has fet clic a: ' + e.target.textContent);
62     }
63   });
64
65   // Afegir noves tasques (també reaccionaran automàticament)
66   botoAfegir.addEventListener('click', function() {
67     comptador++;
68     const novaTasca = document.createElement('li');
69     novaTasca.textContent = 'Tasca ' + comptador;
70     llista.appendChild(novaTasca);
71   });
72 </script>
73
74 </body>
75 </html>
```

Aquí, encara que afegim nous `<li>` amb JavaScript, **el mateix gestor continuarà funcionant**, perquè està lligat al `<ul>` pare (llista), i fa servir `e.target` per saber exactament què s'ha clicat.

1.2.4 Cancel·lar esdeveniments i comportaments per defecte

Alguns esdeveniments tenen comportaments predeterminats (com enviar un formulari o seguir un enllaç). Podem evitar-ho així:

```
1 <html lang="ca">
2 <head>
3   <meta charset="utf-8">
```

```
4 <title>Cancel·lar esdeveniments i comportaments per defecte</title>
5 <style>
6   body {
7     font-family: system-ui, -apple-system, sans-serif;
8     padding: 20px;
9   }
10  form {
11    margin-top: 12px;
12    padding: 12px;
13    background: #f3f4f6;
14    border-radius: 8px;
15    max-width: 400px;
16  }
17  input[type="text"] {
18    padding: 8px;
19    border-radius: 6px;
20    border: 1px solid #ccc;
21    width: 100%;
22    margin-bottom: 8px;
23  }
24  button {
25    padding: 8px 12px;
26    border-radius: 6px;
27    border: 1px solid #ccc;
28    cursor: pointer;
29    background: #e5e7eb;
30  }
31  a {
32    color: #2563eb;
33    text-decoration: underline;
34  }
35 </style>
36 </head>
37 <body>
38
39 <h2>Cancel·lar esdeveniments i comportaments per defecte</h2>
40
41 <p>
42   Per defecte, alguns elements HTML (com els formularis i els
43   enllaços)
44   tenen comportaments automàtics: enviar dades o obrir una
45   nova pàgina.
46   Amb <em>event.preventDefault()</em> podem <strong>cancel·lar
47   </strong> aquesta acció i gestionar-la nosaltres amb
48   JavaScript.
49 </p>
50
51 <h3>Exemple 1: Formulari</h3>
52 <form id="formulari">
53   <label for="nom">Nom:</label>
54   <input type="text" id="nom" placeholder="Escriu el teu nom
55   ...">
56   <button type="submit">Enviar</button>
57 </form>
58
59 <p id="missatge"></p>
60
61 <h3>Exemple 2: Enllaç</h3>
62 <p>
63   <a href="https://www.wikipedia.org" id="enllac">Anar a
64   Wikipedia</a>
65 </p>
```

```
60
61 <script>
62   // Formulari: bloquegem l'enviament per defecte
63   const form = document.getElementById('formulari');
64   const missatge = document.getElementById('missatge');
65
66   form.addEventListener('submit', function(e) {
67     e.preventDefault(); // Evita que el navegador envii el
        formulari
68     const nom = document.getElementById('nom').value.trim();
69
70     if (nom === '') {
71       missatge.textContent = 'Cal escriure un nom abans d'
        enviar.';
72       missatge.style.color = 'red';
73     } else {
74       missatge.textContent = 'Formulari controlat via
        JavaScript. Benvingut/da, ${nom}!';
75       missatge.style.color = 'green';
76     }
77   });
78
79   // Enllaç: evitem que redirigeixi i mostrem un missatge
        propi
80   const enllac = document.getElementById('enllac');
81
82   enllac.addEventListener('click', function(e) {
83     e.preventDefault(); // Cancel·la el comportament per
        defecte (anar a la web)
84     alert('Has clicat l'enllaç, però la navegació ha estat
        bloquejada amb preventDefault().');
85   });
86 </script>
87
88 </body>
89 </html>
```

Això és imprescindible per poder validar dades abans d'enviar-les, o per fer enviaments AJAX, per exemple.

```
1 export default function saluda(nom) {  
2   return 'Hola, ${nom}!';  
3 }
```

Ara podríem voler importar-lo a `main.js`. Per importar un *export* per defecte **no cal usar claus**:

```
1 import saluda from './salutacio.js';  
2  
3 console.log(saluda('Anna')); // "Hola, Anna!"
```

Només pot haver-hi **un export default per fitxer**.

3.1.3 Estructura d'un projecte modular

Un projecte amb mòduls pot tenir una estructura així:

```
1 /projecte  
2 |  
3 |-- /src  
4 |   |-- utils.js  
5 |   |-- dades.js  
6 |   |-- main.js  
7 |  
8 |-- index.html
```

El fitxer `utils.js` podria contenir funcions generals, `dades.js` podria tenir informació o classes, i `main.js` serà el punt d'entrada que importarà i coordinarà la resta.

La programació amb mòduls té diverses avantatges:

- **Organització:** el codi queda separat per funcionalitats.
- **Reutilització:** podem importar el mateix mòdul en diversos llocs.
- **Mantenibilitat:** és més fàcil actualitzar o corregir errors.
- **Evitació de conflictes:** cada mòdul té el seu espai de noms (*scope*).
- **Execució controlada:** cada mòdul s'executa només una vegada.

Aquestes són algunes bones pràctiques en programació modular:

- Cada mòdul ha de tenir una **responsabilitat clara** (no s'hauria de posar tot el codi en un sol fitxer).
- Els noms de les funcions han de ser **coherents i descriptius**.

- Usar **exportacions nominals** (`export { ... }`) quan hi ha moltes funcions.
- Usar **exportació per defecte** quan el mòdul només fa una tasca principal.
- Evitar variables globals: declarar tot dins del mòdul o dins de funcions.

3.2 Empaquetadors de Javascript

Quan treballem amb mòduls en un projecte petit, és fàcil mantenir-los: només hem d'afegir `<script type="module">` al nostre HTML.

Però quan el projecte creix i tenim **centenars de fitxers modulars**, el navegador ha de fer moltes peticions separades per descarregar-los tots, cosa que **alenteix molt la càrrega** de la pàgina.

Els **empaquetadors de JavaScript** (*bundlers*) solucionen aquest problema combinant tots els nostres mòduls i dependències en **un únic fitxer optimitzat**, preparat per producció.

Un **empaquetador** agafa tots els teus fitxers `.js`, `.css` i altres recursos, els **analitza, combina i minifica**, i genera un o pocs fitxers optimitzats per al navegador.

3.2.1 Els empaquetadors més utilitzats

En la taula 4.1 mostrem una llista dels empaquetadors disponibles. Nosaltres, en les diferents activitats i exercicis, optarem per Vite.

TAULA 3.1. Empaquetadors

Nom	Característiques principals
Webpack	El més complet i flexible. Permet empaquetar JS, CSS, imatges, fonts i fins i tot HTML. Ideal per a projectes grans.
Parcel	No requereix configuració inicial (<i>zero config</i>). Ideal per començar amb projectes moderns sense complicacions.
Vite	Creat pels autors de Vue. Molt ràpid gràcies a l'ús de mòduls nadius d'ECMAScript i servidor de desenvolupament ultraràpid.
Rollup	Especialitzat en mòduls ES6. Genera fitxers molt nets i lleugers, perfecte per llibreries i components reutilitzables.

3.2.2 Exemple pràctic amb Vite

Suposem que volem crear un projecte modern amb mòduls. Amb Vite, podem fer-ho molt fàcilment:

1. Obrim un terminal i executem:

```
1 npm create vite@latest projecte-vite
```

2. Triem el tipus de projecte (per exemple, “Vanilla” per JavaScript pur).

3. Entrem a la carpeta:

```
1 cd projecte-vite
```

4. Instal·lem les dependències:

```
1 npm install
```

5. I arranquem el servidor local:

```
1 npm run dev
```

Vite utilitza directament els **mòduls ES6** del navegador durant el desenvolupament, sense necessitat d’empaquetar res.

Només crea un **fitxer empaquetat i optimitzat** quan fem el desplegament final (npm run build).

3.2.3 Estructura d’un projecte Vite

Un cop s’ha creat un projecte amb Vite, la seva estructura d’arxius típica pot ser la següent:

```
1 /projecte-vite
2 |
3 |-- /node_modules          # Conté les dependències instal·
   lades
4 |-- /public                 # Conté fitxers públics com el
   favicon o el HTML base
5 |   |-- index.html          # Fitxer HTML principal
6 |
7 |-- /src                    # Conté el codi font del projecte
8 |   |-- main.js              # Punt d’entrada de l’aplicació
   JavaScript
9 |   |-- style.css            # Arxiu CSS
10 |
11 |-- package.json            # Conté les dependències del
   projecte i configuració de npm
12 |-- vite.config.js          # Configuració de Vite
```

```
13 | -- /dist          # Conté la versió final del  
    | projecte, optimitzada per producció
```

Tant si utilitzem Vite com qualsevol altre empaquetador trobarem una estructura d'arxius força similar. Aquesta estructura pot variar, sobretot pot créixer si el projecte és gran, però a grans trets sempre veurem els projectes estructurats de forma similar.

Anem a analitzar cadascun dels arxius i carpetes principals:

- **/node_modules:** Aquesta carpeta conté totes les dependències del projecte que han estat instal·lades per **npm**. Aquesta carpeta és creada quan s'executa **npm install** i inclou totes les biblioteques i mòduls que el projecte necessita per funcionar.
- **/public:** Aquesta carpeta conté els arxius públics que el servidor utilitzarà. Generalment, inclou el fitxer **index.html**, que és el punt d'entrada de l'aplicació, així com altres arxius estàtics (com imatges o fonts).
- **/src:** Aquesta carpeta conté els fitxers de codi font de l'aplicació. És on es defineixen els mòduls JavaScript, les funcions d'operació (com les matemàtiques, en el cas de l'exemple), els arxius CSS per a l'estil, etc.
- **package.json:** Aquest fitxer és essencial per a la gestió de dependències de Node.js. Conté les següents seccions:
 - **dependencies:** Les biblioteques i mòduls que el projecte necessita per a funcionar.
 - **devDependencies:** Les eines que es necessiten només durant el desenvolupament (per exemple, Vite o altres eines de compilació).
 - **scripts:** S'hi defineixen comandes com **npm run build** o **npm start**, que són les que es poden executar des de la línia de comandes per realitzar tasques automàtiques com la construcció o el desenvolupament.
- **vite.config.js:** Aquest fitxer conté la configuració específica de Vite per a personalitzar l'entorn de desenvolupament o producció. Aquí es poden definir, per exemple, les rutes d'entrada, optimitzacions, etc.
- **/dist:** Aquesta carpeta és generada quan es fa **npm run build** i conté la versió optimitzada per a producció del projecte. És on es col·loquen els fitxers minificats, empaquetats i llestos per ser servits en un entorn de producció.

3.2.4 Per què fem servir un empaquetador?

Els empaquetadors no només uneixen fitxers, també fan moltes altres tasques automàtiques:

- **Minificació:** eliminen espais i comentaris per reduir la mida dels fitxers.
- **Transpilació:** converteixen codi modern (ES6, ES7, etc.) a versions compatibles amb navegadors antics.
- **Gestió de dependències:** permeten importar llibreries de `node_modules` amb `import`.
- **Hot Reload:** actualitzen automàticament la pàgina quan guardem canvis en el codi.
- **Optimització per producció:** creen fitxers finals més petits, amb nom únic i versions “catxejables”.

3.2.5 Cas pràctic de projecte amb Vite

Per acabar d’entendre bé els punts anteriors cal posar-ho en pràctica, és a dir, crear un projecte des de zero amb un empaquetador i veure la seva utilitat.

Imaginem un **cas pràctic**: volem crear una **calculadora modular amb Vite**.

Suposem la següent **cronologia procedimental**:

1. Crearem un nou projecte calculadora-modular amb Vite.
2. Eliminarem els fitxers que no ens interessin del projecte de mostra i reaprofitarem l’arxiu d’estils `style.css`
3. Crearem tres mòduls:
 - `operacions.js` → que exporta suma, resta, multiplica, divideix.
 - `interficie.js` → que s’encarregarà de llegir els camps d’un formulari i mostrar el resultat. Importarà les operacions de `operacions.js`
 - `main.js` → serà el punt d’entrada que importa `interficie.js` i `style.css`
4. Afegirem un formulari senzill a `index.html` per introduir dos números i triar l’operació.
5. Executarem `npm run dev` i comprovarem el funcionament.
6. Finalment, executa `npm run build` i examinarem com Vite empaqueta tot el projecte.

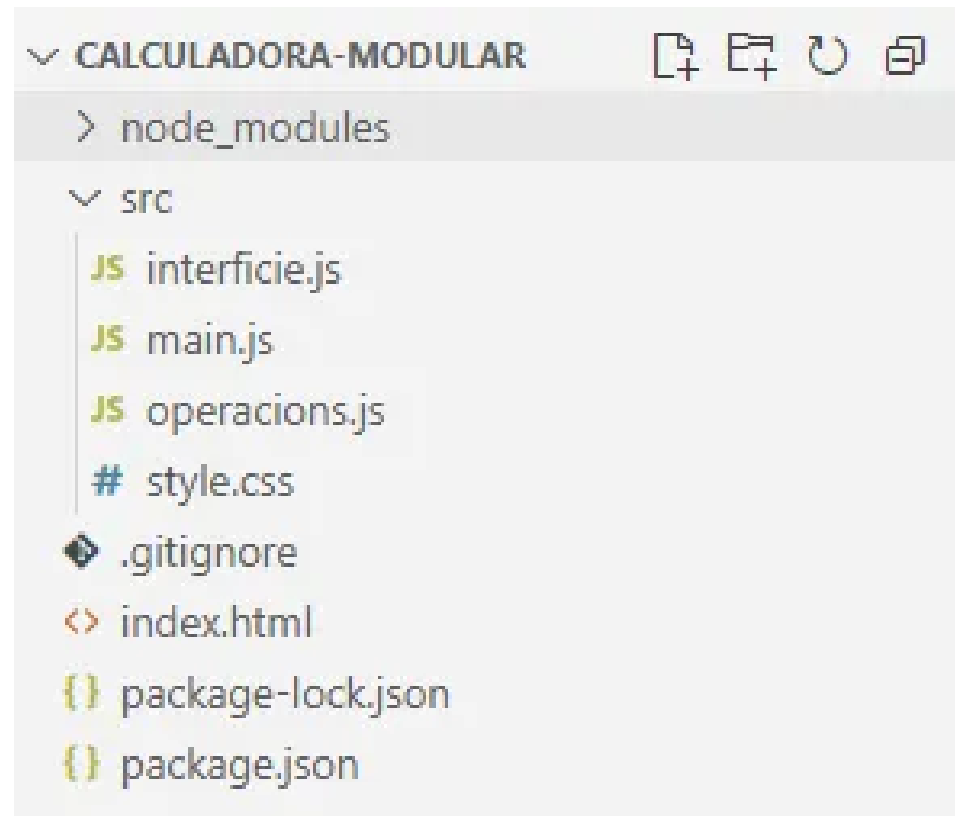
Desenvoluparem el projecte seguint els punts de la cronologia:

1. Crear nou projecte:

```
1 npm create vite@latest calculadora-modular
```


2. L'estructura del projecte pot ser semblant a aquesta (figura 5.1):

FIGURA 3.1. Estructura del projecte



Fitxer style.css:

```
1 :root { line-height: 1.5; }
2 #resultat strong { font-weight: 600; }
```

3. Creació dels mòduls:

Fitxer operacions.js:

```
1 // Funcions pures d'operacions matemàtiques bàsiques
2 export function suma(a, b) { return a + b; }
3 export function resta(a, b) { return a - b; }
4 export function multiplica(a, b) { return a * b; }
5 export function divideix(a, b) {
6   if (b === 0) return "No es pot dividir entre zero.";
7   return a / b;
8 }
```

Fitxer interfície.js:

```
1 import { suma, resta, multiplica, divideix } from "./operacions.
  js";
2
3 function formatNumero(n) {
4   // Evita resultats com 0.30000000000000004
5   const fix = Number.isInteger(n) ? n.toString() : n.toFixed(10)
6   ;
7   return parseFloat(fix).toString();
8 }
```

```
7 }
8
9 export function inicialitzaInterficie() {
10   const form = document.getElementById("calc-form");
11   const aInput = document.getElementById("a");
12   const bInput = document.getElementById("b");
13   const opSelect = document.getElementById("op");
14   const resultat = document.getElementById("resultat");
15
16   form.addEventListener("submit", (e) => {
17     e.preventDefault();
18     const a = parseFloat(aInput.value);
19     const b = parseFloat(bInput.value);
20     const op = opSelect.value;
21
22     try {
23       let valor;
24       switch (op) {
25         case "suma": valor = suma(a, b); break;
26         case "resta": valor = resta(a, b); break;
27         case "multiplica": valor = multiplica(a, b); break;
28         case "divideix": valor = divideix(a, b); break;
29         default: throw new Error("Operació desconeguda");
30       }
31       resultat.textContent = "";
32       resultat.innerHTML = `Resultat: <strong>${formatNumero(
33         valor)}</strong>`;
34     } catch (err) {
35       resultat.textContent = err.message || "S'ha produït un
36         error.";
37     }
38   });
39 }
```

Fitxer main.js:

```
1 import "./style.css";
2 import { inicialitzaInterficie } from "./interficie.js";
3
4 // Inicia l'app quan el DOM està llest
5 if (document.readyState === "loading") {
6   document.addEventListener("DOMContentLoaded",
7     inicialitzaInterficie);
8 } else {
9   inicialitzaInterficie();
10 }
```

4. Creació d'un formulari:

Fitxer index.html:

```
1 <!doctype html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
6       scale=1.0" />
7     <title>Calculadora modular amb Vite</title>
8   </head>
9   <body>
```

```
9      <div id="app" style="max-width: 640px; margin: 2rem auto;  
      font-family: system-ui, -apple-system, Segoe UI, Roboto,  
      Helvetica, Arial, sans-serif;">  
10     <h1>Calculadora modular</h1>  
11     <form id="calc-form" style="display:flex; gap:.5rem; flex-  
      wrap: wrap; align-items: center;">  
12       <input type="number" id="a" required step="any"  
        placeholder="Primer número" style="flex:1; min-width:  
        8rem; padding:.5rem;">  
13       <select id="op" style="padding:.5rem;">  
14         <option value="suma">+</option>  
15         <option value="resta">-</option>  
16         <option value="multiplica">×</option>  
17         <option value="divideix">÷</option>  
18       </select>  
19       <input type="number" id="b" required step="any"  
        placeholder="Segon número" style="flex:1; min-width:  
        8rem; padding:.5rem;">  
20       <button type="submit" style="padding:.6rem 1rem;">  
        Calcula</button>  
21     </form>  
22     <p id="resultat" aria-live="polite" style="margin-top:1rem  
      ; font-size:1.125rem;"></p>  
23   </div>  
24   <script type="module" src="/src/main.js"></script>  
25 </body>  
26 </html>
```

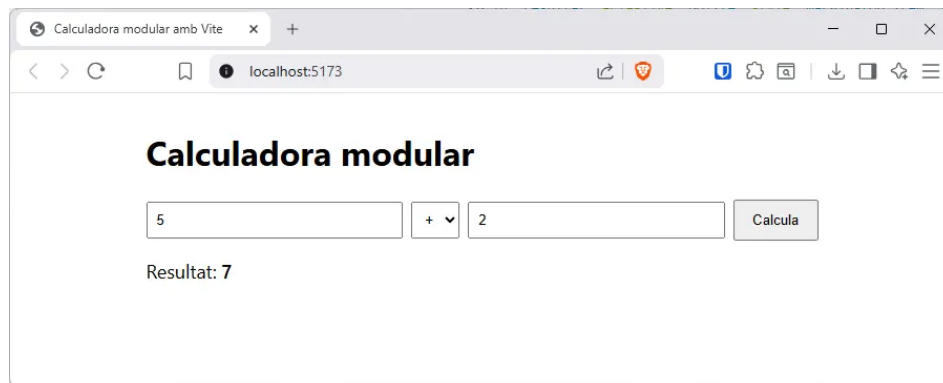
5. Execució i prova en mode de desenvolupament:

```
1 npm run dev
```

Podem veure el resultat de l'execució en la línia d'ordres (figura 5.4), i obrim el navegador per veure com s'executa (figura 5.2).

FIGURA 3.2. Línia de comandes

```
VITE v7.1.11  ready in 431 ms  
  
→ Local:    http://localhost:5173/  
→ Network:  use --host to expose  
→ press h + enter to show help
```

FIGURA 3.3. Execució en el navegador

6. Execució i prova en mode de producció:

```
1 npm run build
```

Ja tenim el projecte empaquetat dins la carpeta dist (figura 5.3):

FIGURA 3.4. carpeta "dist/"

Vite ja haurà creat les rutes correctes per trobar els fitxer `.css` i `.js`. Fixem-nos que ha posat uns noms a l'atzar, però l'empaquetador ja haurà creat les rutes correctes al fitxer `index.html`. El projecte ja està llest per ser posat en producció.

Quan es fa el **procés d'empaquetatge** (`npm run build`), l'eina de construcció (com Vite, Webpack, etc.) empaqueta, optimitza i minimitza tots els fitxers del projecte (JavaScript, CSS, imatges, etc.) i els col·loca dins de la carpeta **dist** (abreviatura de *distribution*). Aquest conjunt de fitxers és el que es desplegarà al servidor per ser servit als usuaris.